COMPUTER VISION PROJECT REPORT

Real-Time Object Detection using YOLO

YOLOv8	Python 3	Detection	OpenCV
Framework	Language	Task	Interface

Repository anshusingh16624-a11y/Computer_Vision_Project	**Date** March 30, 2026	**Domain** Computer Vision / AI

1. Executive Summary

This report presents a comprehensive technical analysis of the **Computer Vision Project** hosted on GitHub under the username *anshusingh16624-a11y*. The project focuses on **real-time object detection** leveraging the **YOLO (You Only Look Once)** deep learning architecture — one of the most widely adopted frameworks for single-stage, high-speed object detection in the field of computer vision.

The system is designed to detect and localize multiple objects within images or video streams simultaneously, drawing bounding boxes and class labels in real time. YOLO's unified detection pipeline enables the model to process an entire image in a single forward pass, making it exceptionally suitable for real-world deployment scenarios that demand both accuracy and speed.

Key highlights of the project include its use of a pre-trained or custom-trained YOLOv8 model, integration with OpenCV for camera/video input handling, Python as the core programming language, and PyTorch as the underlying deep learning backend.

Core Capabilities at a Glance

Real-Time Detection	Process live video feed or static images at high frame rates using YOLO.
Multi-Class Recognition	Detect and classify multiple object categories simultaneously.

Bounding Box Visualization	Draw labeled bounding boxes with confidence scores on output frames.
Configurable Confidence Threshold	Filter weak detections by tuning detection sensitivity.
Modular Codebase	Clean Python modules for training, inference, and evaluation.

2. Project Overview

2.1 Background & Motivation

Object detection is a fundamental problem in computer vision with applications spanning autonomous vehicles, surveillance systems, medical imaging, robotics, and retail analytics. Traditional detection methods relied on hand-crafted features and multi-stage pipelines, which were slow and difficult to scale. The emergence of deep convolutional neural networks and end-to-end detection frameworks like YOLO transformed the landscape.

YOLO reformulates detection as a single regression problem — predicting bounding box coordinates and class probabilities directly from image pixels in one evaluation. This project implements and explores this paradigm using the state-of-the-art **YOLOv8** variant developed by Ultralytics.

2.2 Problem Statement

The project aims to build a robust, real-time object detection pipeline that can:

- Accept input from a webcam feed, video file, or static images.
- Detect objects from one or more categories with high accuracy.
- Display detection results visually with bounding boxes and confidence labels.
- Operate at sufficient frame rates for real-time or near-real-time performance.
- Be easily extendable to custom datasets via fine-tuning.

2.3 Project Scope

In Scope	Object detection inference pipeline, real-time video processing, bounding box rendering, confidence thresholding, COCO pre-trained weights, custom dataset training pipeline.
Out of Scope	Instance segmentation, keypoint estimation, 3D object localization, deployment to mobile/edge devices (unless separate branch exists).

3. Technical Architecture

3.1 YOLO Architecture Overview

YOLO (You Only Look Once) divides the input image into an **S × S grid**. Each grid cell predicts *B* bounding boxes and *C* class probabilities. The output is a tensor of shape **S × S × (B × 5 + C)**, where 5 = [x, y, w, h, confidence]. YOLOv8 builds on this foundation with a CSPDarknet backbone, PANet neck, and decoupled detection head.

Component	Description
Backbone (CSPDarknet)	Extracts rich multi-scale feature representations from input images using residual connections and cross-stage partial networks.
Neck (PANet)	Path Aggregation Network aggregates features across multiple scales to improve detection of both small and large objects.
Detection Head	Decoupled head for classification and regression — predicts class probabilities and bounding box deltas independently.
Anchor-Free Design	YOLOv8 uses an anchor-free approach, simplifying training and reducing hyperparameter tuning.
Loss Function	Combines CIoU loss (bounding box), Binary Cross-Entropy (classification), and Distribution Focal Loss.
NMS Post-Processing	Non-Maximum Suppression removes duplicate detections, keeping only the highest-confidence predictions per object.

3.2 Pipeline Flow

The end-to-end inference pipeline follows these sequential stages:

1. Input Acquisition

Capture frame from webcam, video file, or load a static image using OpenCV (cv2.VideoCapture / cv2.imread).

2. Pre-processing

Resize image to model input size (e.g., 640×640), normalize pixel values to [0, 1], convert BGR to RGB.

3. Model Inference

Pass the tensor through the YOLOv8 network (Ultralytics API or raw PyTorch forward pass).

4. Post-processing

Apply confidence threshold filtering, then Non-Maximum Suppression (NMS) to eliminate redundant boxes.

5. Visualization

Draw bounding boxes, class labels, and confidence scores onto the original frame using OpenCV drawing functions.

6. Output

Display annotated frame in a window, write to output video file, or save annotated image.

3.3 Key Code Structure

Typical project layout for a YOLO-based detection project:

```
Computer_Vision_Project/
■■■ data/ # Dataset images & labels (YOLO format .txt)
■■■ models/ # Model weights (.pt) and configs
■■■ runs/ # Training & validation output (auto-generated)
■■■ detect.py # Main inference script
■■■ train.py # Custom training pipeline
■■■ utils.py # Helper functions (NMS, drawing, metrics)
■■■ requirements.txt # Python dependencies
■■■ README.md # Project documentation
```

4. Technology Stack

Technology	Version / Variant	Purpose
Python	3.9 – 3.11	Core programming language
YOLOv8	Ultralytics 8.x	Object detection model & training API
PyTorch	≥ 2.0	Deep learning backend for YOLO inference
OpenCV	4.x	Image/video I/O and visualization
NumPy	1.24+	Numerical operations on tensors/arrays
Matplotlib	3.x	Loss curves and evaluation plots
CUDA (opt.)	11.8 / 12.x	GPU acceleration for training & inference
pip / conda	—	Dependency and environment management

Installation & Setup

```
# Clone the repository
git clone https://github.com/anshusingh16624-a11y/Computer_Vision_Project.git
cd Computer_Vision_Project

# Install dependencies pip
install -r requirements.txt

# Core packages
pip install ultralytics opencv-python torch torchvision
```

5. Implementation Details

5.1 Inference / Detection

The primary inference script loads a YOLO model (pre-trained on COCO or custom weights), processes each input frame, and overlays detection annotations:

```
from ultralytics import YOLO
import cv2

model = YOLO('yolov8n.pt') # Load nano model
cap = cv2.VideoCapture(0) # Webcam source

while cap.isOpened():
ret, frame = cap.read()
if not ret: break
results = model(frame, conf=0.5) # Inference with 50%
threshold annotated = results[0].plot() # Draw boxes on frame
cv2.imshow('YOLOv8 Detection', annotated) if cv2.waitKey(1) &
0xFF == ord('q'): break

cap.release()
cv2.destroyAllWindows()
```

5.2 Custom Training

For domain-specific detection, the model can be fine-tuned on a custom dataset annotated in YOLO format (one .txt label file per image, normalized coordinates):

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # Start from pre-trained weights
model.train(
data='data/custom.yaml', # Dataset
config epochs=50, imgsz=640, batch=16,
device='cuda', # GPU training
name='custom_run'
)
```

5.3 Evaluation Metrics

Model performance is evaluated using standard object detection metrics:

mAP@0.5	Mean Average Precision at IoU threshold 0.50 — primary metric.
mAP@0.5:0.95	Average mAP across IoU thresholds 0.50–0.95 (COCO standard).

Precision (P)	Fraction of predicted boxes that are correct.
Recall (R)	Fraction of ground-truth boxes that were detected.
F1 Score	Harmonic mean of precision and recall.
Inference Speed	FPS (frames per second) measured on target hardware.

6. Results & Performance

6.1 Benchmark Performance (COCO Dataset)

The following table summarises typical YOLOv8 performance on the MS-COCO val2017 benchmark across model variants. Actual project results may vary based on hardware and configuration.

Model	Parameters	mAP@0.5	mAP@0.5:0.95	Speed (A100 GPU)	Speed (CPU)
YOLOv8n	3.2M	52.4%	37.3%	~0.99 ms/img	~80 ms/img
YOLOv8s	11.2M	58.6%	44.9%	~1.20 ms/img	~128 ms/img
YOLOv8m	25.9M	63.6%	50.2%	~1.83 ms/img	~234 ms/img
YOLOv8l	43.7M	66.2%	52.9%	~2.39 ms/img	~375 ms/img
YOLOv8x	68.2M	68.6%	53.9%	~3.53 ms/img	~479 ms/img

Source: Ultralytics YOLOv8 official documentation benchmark results.

6.2 Real-Time Detection Performance

Hardware	Resolution	Approx. FPS	Suitability
NVIDIA RTX 3060 GPU	640×640	85–120 FPS	✓ Real-time
NVIDIA GTX 1650 GPU	640×640	45–70 FPS	✓ Real-time

Intel Core i7 CPU	640×640	6–12 FPS	■ Near real-time
Raspberry Pi 4	320×320	2–4 FPS	✗ Not real-time

7. Applications & Use Cases

Surveillance & Security	Detect intruders, monitor crowd density, identify abandoned objects in CCTV feeds.
Autonomous Vehicles	Real-time detection of pedestrians, vehicles, traffic signs, and road hazards.
Retail Analytics	Count customer footfall, detect product placement compliance, monitor shelf stock.
Medical Imaging	Assist radiologists by detecting anomalies in X-rays or histopathology slides.
Sports Analytics	Track players and ball in broadcast footage for performance analysis.
Industrial Inspection	Detect manufacturing defects, foreign objects, or safety violations on assembly lines.

Agriculture	Monitor crop health, detect pests, count livestock from drone imagery.
Smart Classrooms	Track student engagement, detect raised hands, monitor attendance automatically.

8. Challenges & Limitations

Small Object Detection

YOLO models can struggle with very small or densely packed objects. Mitigation: use higher input resolution (e.g., 1280px) or SAHI (sliced inference).

Occlusion Handling

Partially occluded objects may not be detected reliably. This requires training data with occluded examples.

Domain Shift

Pre-trained COCO weights may not generalize well to specialized domains (e.g., medical, industrial). Fine-tuning on domain data is essential.

Lighting & Blur

Extreme lighting conditions or motion blur degrade detection accuracy. Data augmentation (brightness jitter, blur) helps.

Computational Cost

Real-time inference on CPU is challenging for large model variants. Quantization (INT8) or ONNX export can reduce latency.

Class Imbalance

When training on custom datasets, rare object classes may be underdetected without oversampling or focal loss weighting.

False Positives

Confidence threshold tuning is critical — too low leads to noise, too high misses true detections.

9. Future Enhancements

- Integrate YOLOv9 / YOLOv10 for improved accuracy–speed trade-offs.
- Add instance segmentation (YOLOv8-seg) for pixel-level masks alongside bounding boxes.
- Deploy the model as a REST API using FastAPI or Flask for web-based inference.
- Export to ONNX / TensorRT for optimized edge deployment.
- Implement object tracking (e.g., ByteTrack, DeepSORT) to assign persistent IDs across frames.
- Build a Gradio / Streamlit dashboard for no-code inference and model evaluation.
- Extend to multi-camera setups with spatial calibration for 3D object localization.
- Automate dataset collection and labelling using active learning pipelines.
- Benchmark against RT-DETR and DINO-based detectors for academic comparison.

10. References

[1] Redmon, J. et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection.* CVPR.

[2] Jocher, G. et al. (2023). *Ultralytics YOLOv8.* https://github.com/ultralytics/ultralytics

[3] Lin, T.-Y. et al. (2014). *Microsoft COCO: Common Objects in Context.* ECCV.

[4] Wang, C.-Y. et al. (2023). *YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information.* arXiv.

[5] Bradski, G. (2000). *The OpenCV Library.* Dr. Dobb's Journal of Software Tools.

[6] Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS.

[7] GitHub Repository: https://github.com/anshusingh16624-a11y/Computer_Vision_Project